K8S-13: Kafka Monitoring with Kpow

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 13 of 14 | **Created:** February 2026 | **Last Updated:** 10/15/2026

Overview

Kpow (by Factor House) is an all-in-one toolkit for managing, monitoring, and learning about Apache Kafka clusters. When deployed on Kubernetes alongside Dynatrace, you get deep Kafka observability by combining Kpow's calculated telemetry with Dynatrace's infrastructure monitoring and alerting capabilities.

This notebook covers deploying Kpow on Kubernetes, scraping its Prometheus metrics into Dynatrace, writing DQL queries against Kafka health data, and building alerts for consumer lag and broker issues.

Table of Contents

1. Kpow Architecture and Metrics
2. Deploying Kpow on Kubernetes
3. Configuring Dynatrace Prometheus Scraping
4. Querying Kpow Metrics with DQL
5. Monitoring Kpow as a Kubernetes Workload
6. Alerting on Kafka Health
7. Combining with Native Kafka Monitoring

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring enabled
Kubernetes Cluster	Dynatrace Operator v1.0+ with ActiveGate
Apache Kafka	Running Kafka cluster accessible from K8s
Kpow License	Community or Enterprise license from Factor House
Helm	v3+ for Kpow deployment
Knowledge	Completed K8S-01, K8S-05 (Workload Monitoring)

1. Kpow Architecture and Metrics

What Kpow Provides

Kpow calculates its own telemetry about your Kafka cluster independently of Kafka's built-in JMX metrics. This means:

- **No JMX dependency** — frictionless installation, no broker-side configuration
- **Calculated metrics** — consumer group lag, topic throughput, offset deltas
- **Multi-resource support** — brokers, topics, consumer groups, Kafka Connect, Schema Registry, ksqlDB

Prometheus Metric Endpoints

Kpow exposes OpenMetrics-compatible endpoints when `PROMETHEUS_EGRESS=true`:

Endpoint	Description
`GET /metrics/v1`	All metrics across the system
`GET /metrics/v1/cluster/:cluster-id`	Cluster-specific metrics
`GET /metrics/v1/connect/:connect-id`	Kafka Connect metrics
`GET /metrics/v1/schema/:schema-id`	Schema Registry metrics
`GET /metrics/v1/ksqldb/:ksqldb-id`	ksqlDB metrics

Endpoint	Description
`GET /offsets/v1`	Topic offset data
`GET /group-offsets/v1`	Consumer group offset data
`GET /streams/v1`	Kafka Streams metrics

Key Metrics

Metric	Type	Description
`broker_count`	Gauge	Number of brokers in the Kafka cluster
`topic_count`	Gauge	Number of topics in the cluster
`group_count`	Gauge	Number of consumer groups
`group_offset_lag`	Histogram	Consumer group lag across assignments
`group_state`	Gauge	Consumer group state — the ordinal of Kafka's `ConsumerGroupState` (0=UNKNOWN, 1=PREPARING_REBALANCE, 2=COMPLETING_REBALANCE, 3=STABLE, 4=DEAD, 5=EMPTY)
`broker_end_delta`	Histogram	Delta of end offsets per broker (produced msgs/s)
`topic_urp_total`	Meter	Under-replicated partitions across all topics in the cluster
`acl_count`	Gauge	Number of ACLs in the cluster
`connect_connector_task_state`	Gauge	Connector task state
`connect_connector_running_total`	Gauge	Number of active connectors
`partition_end`	Gauge	Topic partition end offset (from `/offsets/v1`)
`group_assignment_offset`	Gauge	Current consumer position (from `/group-offsets/v1`)

Metric Label Structure

All Kpow metrics use consistent labels:

Label	Description	Example
`domain`	Category of metric	`cluster`, `connect`, `streams`
`id`	Unique identifier	Kafka Cluster ID
`target`	Resource identifier	Consumer group name, topic name
`env`	Environment name	`Production`, `Staging`

> **Note:** Non-compliant characters in Kafka resource names (topics, groups) are converted to underscores to meet Prometheus naming standards.
>
> `group_state` **ordinals come from Kafka, not Kpow.** Kpow's glossary defines the value as *"the ordinal of org.apache.kafka.common.ConsumerGroupState"*, and that enum is declared in the order UNKNOWN, PREPARING_REBALANCE, COMPLETING_REBALANCE, STABLE, DEAD, EMPTY (then ASSIGNING, RECONCILING). A dead group is **4**; 0 means unknown.
>
> **Sources:** Kpow metrics glossary (Factor House) — *"The state of the consumer group, where the value is the ordinal of org.apache.kafka.common.ConsumerGroupState"*, ConsumerGroupState.java (Apache Kafka GitHub).

2. Deploying Kpow on Kubernetes

Helm Chart Installation

Factor House provides an official Helm chart for Kubernetes deployment.

Step 1: Add the Helm repository

```
helm repo add factorhouse https://charts.factorhouse.io
helm repo update
```

Step 2: Create the namespace and license secret

```
kubectl create namespace kpow

# Create a secret for your Kpow license
kubectl create secret generic kpow-license \
  --namespace kpow \
  --from-literal=LICENSE_ID=your-license-id \
  --from-literal=LICENSE_CODE=your-license-code
```

Step 3: Create a `values.yaml` with Prometheus enabled and Dynatrace annotations

```
# kpow-values.yaml
replicaCount: 1

env:
  # Kafka cluster connection
  BOOTSTRAP: "kafka-broker-0.kafka:9092,kafka-broker-1.kafka:9092"

  # Enable Prometheus metrics export
  PROMETHEUS_EGRESS: "true"

  # Optional: Basic auth for metrics endpoints
  # PROMETHEUS_USERNAME: "metrics"
  # PROMETHEUS_PASSWORD: "&lt;from-secret&gt;"

# Kpow serves on port 3000 by default
service:
  port: 3000

resources:
  requests:
    cpu: 500m
    memory: 1Gi
  limits:
    cpu: 2000m
    memory: 2Gi

# Dynatrace Prometheus scrape annotations
podAnnotations:
  metrics.dynatrace.com/scrape: "true"
  metrics.dynatrace.com/port: "3000"
  metrics.dynatrace.com/path: "/metrics/v1"
```

Step 4: Install Kpow

```
helm install kpow factorhouse/kpow \
  --namespace kpow \
  --values kpow-values.yaml
```

Step 5: Verify deployment

```
# Check pods are running
kubectl get pods -n kpow

# Verify Prometheus endpoints are accessible
kubectl port-forward svc/kpow 3000:3000 -n kpow
curl http://localhost:3000/metrics/v1
```

Resource Sizing Guidelines

Kafka Cluster Size	Kpow CPU	Kpow Memory	Notes
Small (< 10 topics)	500m	1Gi	Dev/test environments
Medium (10-100 topics)	1000m	2Gi	Standard production
Large (100+ topics)	2000m	4Gi	High-throughput clusters
Multi-cluster	2000m	4Gi	Monitoring multiple clusters

3. Configuring Dynatrace Prometheus Scraping

How Dynatrace Scrapes Prometheus Metrics

Dynatrace collects metrics from any Kubernetes pod annotated with `metrics.dynatrace.com/scrape: "true"`. The Dynatrace Operator's ActiveGate connects directly to annotated pods and scrapes their Prometheus endpoints, enriching the metrics with Kubernetes topology information.

> **Important:** Dynatrace recommends an ActiveGate running **inside** the monitored cluster. One running outside *"won't be able to scrape any Prometheus endpoints on pods which require authentication (such as RBAC or client authentication)"*.
>
> **ActiveGate scraping has hard limits — consider the OpenTelemetry Collector for new setups.** The docs: *"This page describes the ActiveGate Kubernetes module's Prometheus integration, which has hard limits of 1,000 exporter pods, 1,000 metrics per pod, and 500,000 metric data points per pod. For new deployments, and for any setup approaching these limits, we recommend scraping Prometheus with the OpenTelemetry Collector instead."* A single Kpow pod is far below these limits, so the annotation route below works, but on a cluster that already scrapes many exporters, add Kpow to an

> OpenTelemetry Collector scrape instead — it can reuse the same `metrics.dynatrace.com/scrape` annotations.

Required Pod Annotations

Annotation	Required	Default	Description
`metrics.dynatrace.com/scrape`	Yes	-	Set to `"true"` to enable scraping
`metrics.dynatrace.com/port`	No	First TCP port	Port where metrics are exposed
`metrics.dynatrace.com/path`	No	`/metrics`	Path to the metrics endpoint
`metrics.dynatrace.com/secure`	No	`"false"`	Use HTTPS for scraping
`metrics.dynatrace.com/insecure_skip_verify`	No	`"false"`	Skip TLS certificate verification (self-signed certs); honoured only by an in-cluster ActiveGate
`metrics.dynatrace.com/filter`	No	-	JSON filter, for example `{"mode": "include", "names": ["broker_*", "group_*"]}`; `*` wildcards allowed

Applying Annotations to Kpow

If you deployed Kpow via Helm with `podAnnotations` (as shown above), the annotations are already applied. To verify:

```
kubectl get pods -n kpow -o jsonpath='{.items[0].metadata.annotations}' |
python3 -m json.tool
```

To add annotations to an existing Kpow deployment:

```
kubectl annotate pods -n kpow -l app=kpow \
  metrics.dynatrace.com/scrape="true" \
  metrics.dynatrace.com/port="3000" \
  metrics.dynatrace.com/path="/metrics/v1"
```

Scraping Multiple Endpoints

Kpow exposes several metric endpoints. The primary `/metrics/v1` endpoint includes all cluster metrics. For offset-specific data, you can deploy a sidecar or use additional scrape targets.

Topic offsets (`partition_start` , `partition_end` , `topic_end_sum`) come from `/offsets/v1` , and group-assignment offsets (`group_assignment_offset` and related) from `/group-offsets/v1` . Each annotation set names one path. To scrape the offsets endpoint separately, add a second annotated service, and a third for `/group-offsets/v1` if you need it:

```
# Additional Service for offset metrics
apiVersion: v1
kind: Service
metadata:
  name: kpow-offsets
  namespace: kpow
  annotations:
    metrics.dynatrace.com/scrape: "true"
    metrics.dynatrace.com/port: "3000"
    metrics.dynatrace.com/path: "/offsets/v1"
spec:
  selector:
    app: kpow
  ports:
    - port: 3000
      targetPort: 3000
```

Enabling Prometheus Monitoring in Dynatrace

Ensure Prometheus scraping is enabled in the cluster's Kubernetes monitoring settings:

1. Open the cluster's monitoring settings
2. Enable **Monitor Kubernetes namespaces, services, workloads, and pods**
3. Enable **Monitor annotated Prometheus exporters** — the settings schema notes *"Prometheus metrics in kubernetes environments are subject to licensing."*

> **Note:** Dynatrace ingests Prometheus Counter, Gauge, Histogram and Summary types. Kpow's glossary also lists *meter* types (for example `topic_urp_total`); check those arrive before building on them.
>
> **Sources:** Monitor Prometheus metrics on Kubernetes (DT docs) — *"For new deployments, and for any setup approaching these limits, we recommend scraping Prometheus with the OpenTelemetry Collector instead."*, Kubernetes monitoring settings schema (DT docs) — *"Prometheus metrics in kubernetes environments are subject to licensing."*

```
// Verify Kpow metrics are being ingested
timeseries brokerCount = avg(broker_count), from:-1h
| fieldsAdd avgBrokers = arrayAvg(brokerCount)
```

4. Querying Kpow Metrics with DQL

Once Dynatrace scrapes Kpow's Prometheus endpoints, the metrics are available in Grail for DQL queries. Prometheus metrics are queryable using the `timeseries` command.

Cluster Health Overview

```
// Kafka cluster overview: brokers, topics, consumer groups
timeseries brokers = avg(broker_count), from:-1h
| append [
    timeseries topics = avg(topic_count), from:-1h
  ]
| append [
    timeseries groups = avg(group_count), from:-1h
  ]
```

Consumer Group Lag

```
// Consumer group lag over time
timeseries lag = avg(group_offset_lag), from:-1h, by:{target}
| fieldsAdd avgLag = arrayAvg(lag)
| sort avgLag desc
| limit 10
```

Consumer Group State

```
// Consumer group state — the value is the ordinal of Kafka's
ConsumerGroupState:
// 0=UNKNOWN, 1=PREPARING_REBALANCE, 2=COMPLETING_REBALANCE, 3=STABLE, 4=DEAD,
5=EMPTY
timeseries state = max(group_state), from:-1h, by:{target}
| fieldsAdd currentState = arrayLast(arrayRemoveNulls(state))
| fieldsAdd stateLabel = if(currentState == 3, then: "STABLE",
    else: if(currentState == 4, then: "DEAD",
    else: if(currentState == 5, then: "EMPTY",
    else: if(currentState == 1 or currentState == 2, then: "REBALANCING",
    else: if(currentState == 0, then: "UNKNOWN", else: "OTHER")))))
| fields target, currentState, stateLabel
| sort stateLabel asc
```

Under-Replicated Partitions

```
// Under-replicated partitions trend (critical health indicator)
timeseries urp = avg(topic_urp_total), from:-6h
| fieldsAdd avgUrp = arrayAvg(urp)
| filter avgUrp &gt; 0
```

Broker Production Rate

```
// Broker production rate (messages produced per broker)
timeseries delta = avg(broker_end_delta), from:-1h, by:{target}
| fieldsAdd avgDelta = arrayAvg(delta)
| sort avgDelta desc
| limit 10
```

Kafka Connect Health

```
// Kafka Connect: tasks per connector that are not running
// Kpow's glossary does not publish the value mapping for
connect_connector_task_state, so use
// the explicit counters instead.
timeseries {
    running = max(connect_connector_task_running_total),
    total = max(connect_connector_task_total)
  }, from:-1h, by:{target}
| fieldsAdd runningNow = arrayLast(arrayRemoveNulls(running)), totalNow =
arrayLast(arrayRemoveNulls(total))
| fieldsAdd notRunning = totalNow - runningNow
| fields target, totalNow, runningNow, notRunning
| sort notRunning desc
```

5. Monitoring Kpow as a Kubernetes Workload

Beyond the Kafka metrics Kpow exposes, you should also monitor Kpow itself as a Kubernetes workload. Kpow runs as a JVM process, so Dynatrace OneAgent provides:

Signal	What It Shows
Pod health	Restarts, OOMKilled events, scheduling issues
CPU/Memory	Resource consumption of the Kpow process
JVM metrics	Heap usage, garbage collection, threads
Logs	Kpow application logs for troubleshooting

Kpow Pod Resource Usage

```
// Kpow pod CPU usage — the pod is the sum of its containers
timeseries cpuUsage = sum(dt.kubernetes.container.cpu_usage, rollup: avg),
from:-1h,
  by:{k8s.namespace.name, k8s.pod.name},
  filter:{k8s.namespace.name == "kpow"}
| fieldsAdd avgCpuMillicores = round(arrayAvg(cpuUsage), decimals: 0)
| fields k8s.namespace.name, k8s.pod.name, avgCpuMillicores
| sort avgCpuMillicores desc
```

```
// Kpow pod memory working set — the pod is the sum of its containers
timeseries memUsage = sum(dt.kubernetes.container.memory_working_set, rollup:
avg), from:-1h,
  by:{k8s.namespace.name, k8s.pod.name},
  filter:{k8s.namespace.name == "kpow"}
| fieldsAdd avgMemMiB = round(arrayAvg(memUsage) / 1048576, decimals: 0)
| fields k8s.namespace.name, k8s.pod.name, avgMemMiB
| sort avgMemMiB desc
```

Kpow Application Logs

```
// Kpow error logs
fetch logs, from:-1h
| filter k8s.namespace.name == "kpow"
| filter loglevel == "ERROR" or loglevel == "WARN"
| fields timestamp, loglevel, content, k8s.pod.name
| sort timestamp desc
| limit 50
```

Kpow Pod Restart History

```
// Kpow pod restarts and events
fetch events, from:-24h
| filter event.provider == "KUBERNETES_EVENT"
| filter k8s.namespace.name == "kpow"
| fields timestamp, dt.kubernetes.event.reason,
dt.kubernetes.event.involved_object.name, dt.kubernetes.event.message
| sort timestamp desc
| limit 20
```

6. Alerting on Kafka Health

Kpow does not provide its own alerting — it delegates to external tools. With Dynatrace ingesting Kpow metrics, you can alert with custom alerts on DQL queries and route the resulting problems with Workflows.

Recommended Alert Thresholds

Alert	Metric	Condition	Severity
Consumer lag spike	`group_offset_lag`	Lag > 10,000 for 5 min	Warning
Consumer lag critical	`group_offset_lag`	Lag > 100,000 for 5 min	Critical
Consumer group dead	`group_state`	State == 4 (DEAD)	Critical
Under-replicated partitions	`topic_urp_total`	URP > 0 for 5 min	Critical
Broker count drop	`broker_count`	Count < expected	Critical
Connector task failure	`connect_connector_task_failed_total`	> 0	Warning
Kpow pod unhealthy	`dt.kubernetes.container.restarts`	> 3 in an hour	Warning

The lag thresholds are community starting points; set them from your consumers' normal lag.

Setting Up Custom Alerts

Build each alert as a custom alert on a DQL `timeseries` query over the Kpow metric — for example `timeseries lag = max(group_offset_lag), by:{target}` with a static threshold of 100,000 for the critical lag alert. From SaaS 1.344 custom alerts are created in **Settings**; on earlier versions, in the Anomaly Detection app (K8S-07 § 7 has the detail and source).

Workflow Integration

Connect metric events to Dynatrace Workflows for automated response:

Trigger	Action
Consumer lag > threshold	Notify Slack channel, create Jira ticket
Broker count drops	Page on-call, trigger runbook
URP > 0 sustained	Alert Kafka admin team

Trigger	Action
Connector task fails	Auto-restart connector via API

Agentic triage

For AI-assisted triage on top of these problems, see the AIOPS series.

> **Tip:** See **WFLOW-01** through **WFLOW-09** for comprehensive Dynatrace Workflow configuration patterns.

7. Combining with Native Kafka Monitoring

Dynatrace's Built-in Kafka Support

Dynatrace provides native Kafka monitoring through OneAgent, which automatically detects Kafka broker processes and collects JMX-based metrics.

The split below is community practice, not a Dynatrace comparison — check the Kafka extension's Hub page and your OneAgent version for what each covers today.

Capability	OneAgent (Native)	Kpow
Broker process metrics	Yes	No
JMX metrics	Yes	No
Consumer group lag	Limited	Yes (detailed)
Topic offset tracking	No	Yes
Kafka Connect monitoring	No	Yes
Schema Registry monitoring	No	Yes
ksqlDB monitoring	No	Yes
Distributed traces	Yes	No
Code-level visibility	Yes	No

Recommended Combination Strategy

Use both together for comprehensive Kafka observability:

Layer	Tool	What It Provides
Infrastructure	Dynatrace OneAgent	Broker process health, JVM metrics, host metrics
Application	Dynatrace OneAgent	Distributed traces through Kafka producers/consumers
Kafka Platform	Kpow → Dynatrace	Consumer lag, topic throughput, Connect/Schema/ksqlDB
Kubernetes	Dynatrace Operator	Pod health, resource usage, events for all components

Dynatrace Apache Kafka Extension

For environments where Kpow is not available, Dynatrace also offers the **Apache Kafka extension** on Dynatrace Hub. This extension provides:

- Broker-level metrics via JMX
- Topic and partition statistics
- Consumer group metrics
- Pre-built dashboards

Install from: **Dynatrace Hub > Apache Kafka**. Check the Hub listing for its current feature set and requirements.

```
// Combine: Kpow consumer lag with OneAgent service traces
// Step 1: Check service-level error rates for Kafka consumers
//
// Field notes — all three fail silently if you get them wrong:
//   span.status_code is the real field (otel.status_code does not exist),
//   its values are lowercase ("error", never "ERROR"),
//   and it is null on successful spans, so derive successes as total - errors.
// Group by dt.service.name: service.name exists only on OpenTelemetry spans.
//
// If this returns nothing, check span.kind before assuming a healthy fleet:
//   fetch spans, from:-1h | summarize c = count(), by:{span.kind}
// A cluster whose Kafka clients are not instrumented emits no "consumer" spans
at
// all — which is a coverage gap, not a clean bill of health.
fetch spans, from:-1h
| filter span.kind == "consumer"
| summarize {
    total = count(),
    errors = countIf(span.status_code == "error")
  }, by:{dt.service.name}
| fieldsAdd successes = total - errors
| fieldsAdd errorRate = round(100.0 * errors / total, decimals: 2)
| sort errorRate desc
| limit 10
```

```
// Check Kafka broker processes detected by OneAgent
fetch dt.entity.process_group_instance
| filter contains(entity.name, "kafka") or contains(entity.name, "Kafka")
| fields entity.name, tags
| sort entity.name asc
| limit 20

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "PROCESS"
//   | filter contains(name, "kafka") or contains(name, "Kafka")
//   | fields name, tags
//   | sort name asc
//   | limit 20
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities than
// the classic entity store; for a pre-migration inventory keep the classic
query above.
// Note: on Smartscape, `tags` is a record of tag key -&gt; value (dictionary:
"Note that rule-based
// tags do not exist in the new model."). Classic rule-based tags such as
dt.owner:... do not appear
// there, so the legacy query and the Smartscape query above do not return the
same tags.
```

Summary

In this notebook, you learned:

- **Kpow architecture** and the Prometheus metrics it exposes for Kafka clusters
- **Kubernetes deployment** of Kpow using the official Helm chart
- **Dynatrace Prometheus scraping** via pod annotations for automatic metric ingestion
- **DQL queries** for consumer lag, group state, under-replicated partitions, and broker production rates
- **Workload monitoring** of the Kpow pod itself (CPU, memory, logs, restarts)
- **Alerting patterns** using custom alerts on DQL queries and Workflows
- **Combining Kpow with native Kafka monitoring** for comprehensive observability

---

Next Steps

Next Notebook	Topic
K8S-11: Multi-Tool Coexistence	Running Dynatrace alongside other monitoring tools
K8S-05: Workload Monitoring	Deep dive into Kubernetes workload observability
WFLOW-01: Workflow Fundamentals	Setting up automated alerting workflows

---

References

- Kpow documentation (Factor House)
- Kpow Prometheus integration (Factor House)
- Kpow metrics glossary (Factor House)
- Kpow Helm charts (Factor House)
- Monitor Prometheus metrics on K8s (DT docs)
- Confluent Cloud (Kafka) extension (DT docs)
- ConsumerGroupState.java (Apache Kafka GitHub)
- Apache Kafka extension (Dynatrace Hub)

- Set up Dynatrace on Kubernetes (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.